

Análisis y Mantenimiento Aplicado - EPN Event Manager

Resumen Ejecutivo

El siguiente documento registra los 4 tipos de mantenimiento de software aplicados al sistema **EPN Event Manager**, un backend diseñado intencionalmente con deuda técnica para que los estudiantes practiquen mantenimiento real de software.

1. MANTENIMIENTO CORRECTIVO

Problema Identificado

Ubicación: `src/modules/events/events.service.ts` - método `registerEvent()`, acción DELETE (líneas 58-66)

Incidencia: El sistema registraba eventos DELETE como exitosos (`{ ok: true }`) pero **nunca persistía los datos en la base de datos**.

Código Antes (Buggy)

```
if (action === 'DELETE') {  
  // BUG: se construye el objeto pero se devuelve éxito antes de persistirlo  
  this.deleteRepo.create({  
    source: dto.source,  
    entity: dto.entity,  
    action: dto.action,  
    title: dto.title,  
    payload: payloadStr,  
    createdAt: localDate,  
  });  
  return { ok: true }; // ❌ Retorna sin hacer await save()  
}
```

Código Después (Corregido)

```
if (action === 'DELETE') {  
  const ev = this.deleteRepo.create({  
    source: dto.source,  
    entity: dto.entity,  
    action: dto.action,  
    title: dto.title,  
    description: dto.description,  
    payload: payloadStr,  
    createdAt: isoDate,  
  });  
  await this.deleteRepo.save(ev); // ✅ Ahora persiste correctamente  
  return { ok: true };  
}
```

Justificación

Este cambio clasifica como **Mantenimiento Correctivo** porque:

- ☒ Corrige un comportamiento incorrecto en tiempo de ejecución
- ☒ Los eventos DELETE ahora se guardan correctamente en BD
- ☒ No cambia el contrato API, solo arregla la implementación interna
- ☒ Elimina una regresión funcional silenciosa

2. MANTENIMIENTO ADAPTATIVO

Problema Identificado

Ubicación: `src/modules/events/events.service.ts` - método `registerEvent()` (línea 30)

Incidencia: Las fechas se guardaban en formato local con `toLocaleString()`, lo que generaba:

- Inconsistencias entre sistemas en diferentes zonas horarias
- Problemas de sincronización entre servidores distribuidos
- Dificultad para comparar eventos de diferentes CRUDs
- Incompatibilidad con estándares internacionales (ISO 8601)

Código Antes

```
// Fecha guardada en formato local, no UTC (debilidad intencional)
const localDate = new Date().toLocaleString();
// Ejemplo: "5/6/2026, 10:30:45 AM" (depende del locale del sistema)
```

Código Después

```
// ADAPTATIVO: Fecha guardada en UTC (ISO 8601) para compatibilidad
const isoDate = new Date().toISOString();
// Resultado: "2026-05-06T10:30:45.123Z" (estándar internacional)
```

Cambios en todas las acciones

- **CREATE** : `recorded_at: isoDate`
- **UPDATE** : `timestamp: isoDate`
- **DELETE** : `createdAt: isoDate`
- **QUERY** : `event_date: isoDate`

Justificación

Este cambio clasifica como **Mantenimiento Adaptativo** porque:

- ☒ Adapta el sistema a un nuevo requisito de compatibilidad global
- ☒ Implementa el estándar ISO 8601 para fechas
- ☒ Permite que múltiples CRUDs en diferentes regiones se sincronicen
- ☒ Facilita la integración con servicios cloud y APIs externas
- ☒ No rompe clientes existentes (cambio transparente en capa de datos)

3. MANTENIMIENTO PERFECTIVO

Problema A: Stats incompleto

Ubicación: `src/modules/events/events.service.ts` - método `getStats()` (línea ~103)

Incidencia: El endpoint `GET /stats` no contaba los eventos de la tabla `query_events`, lo que causaba:

- Métricas incompletas del sistema
- Falsa impresión del volumen real de eventos

Código Antes

```
async getStats(): Promise<object> {  
  const createCount = await this.createRepo.count();  
  const updateCount = await this.updateRepo.count();  
  const deleteCount = await this.deleteRepo.count();  
  // ✗ query_events no se incluye en el total  
  return {  
    create: createCount,  
    update: updateCount,  
    delete: deleteCount,  
    total: createCount + updateCount + deleteCount,  
  };  
}
```

Código Después

```
async getStats(): Promise<object> {  
  const createCount = await this.createRepo.count();  
  const updateCount = await this.updateRepo.count();  
  const deleteCount = await this.deleteRepo.count();  
  // ✓ Ahora incluye query_events  
  const queryCount = await this.queryRepo.count();  
  return {  
    create: createCount,  
    update: updateCount,  
    delete: deleteCount,  
    query: queryCount,  
    total: createCount + updateCount + deleteCount + queryCount,  
  };  
}
```

Problema B: Ordenamiento ineficiente en findAll()

Ubicación: `src/modules/events/events.service.ts` - método `findAll()`

Antes: Ordenaba por strings de fecha heterogéneos sin normalizar nombres de columnas

Después: Normaliza todos los timestamps a `_timestamp` y ordena correctamente

```
// Normaliza nombres de campos de fecha a ISO strings y ordena
const merged = [
  ...creates.map((e) => ({
    ...e,
    _table: 'create_events',
    _timestamp: (e as unknown as Record<string, string>).recorded_at,
  })),
  // ... resto de tablas con _timestamp normalizado
];

// Ordena por timestamp ISO normalizado (ascendente)
merged.sort((a, b) => {
  const ta = (a as unknown as Record<string, string>)._timestamp ?? '';
  const tb = (b as unknown as Record<string, string>)._timestamp ?? '';
  return ta.localeCompare(tb);
});
```

Justificación

Estos cambios **clasifican como Mantenimiento Perfectivo** porque:

- ☒ Mejoran la calidad y completitud de reportes
- ☒ Aumentan la precisión de métricas del sistema
- ☒ Optimizan el ordenamiento de eventos
- ☒ No cambian la funcionalidad, la mejoran
- ☒ Aumentan el valor analítico del sistema

4. MANTENIMIENTO PREVENTIVO

Problema A: Falta de validación de entrada

Ubicación: `src/modules/events/dto/create-event.dto.ts`

Incidencia: El DTO no validaba:

- Campos requeridos vs opcionales
- Longitud máxima de strings
- Tipos de datos

Solución: Agregamos decoradores de `class-validator`

```
import {
  IsString,
  IsNotEmpty,
  IsOptional,
  MaxLength,
} from 'class-validator';

export class CreateEventDto {
  @IsString()
  @IsNotEmpty()
  @MaxLength(50)
  source: string;

  @IsString()
  @IsNotEmpty()
  @MaxLength(100)
  entity: string;

  @IsString()
  @IsNotEmpty()
  @MaxLength(20)
  action: string;

  @IsString()
  @IsNotEmpty()
  @MaxLength(255)
  title: string;

  @IsString()
  @IsOptional()
  @MaxLength(1000)
  description?: string;

  @IsOptional()
  payload?: any;
}
```

Problema B: Falta de global validation pipe

Ubicación: `src/main.ts`

Solución: Agregamos ValidationPipe global

```
import { ValidationPipe } from '@nestjs/common';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  // PREVENTIVO: Validación global para DTOs
  app.useGlobalPipes(new ValidationPipe());
  await app.listen(process.env.PORT ?? 3000);
}
```

Problema C: Health check sin validación real

Ubicación: `src/modules/health/health.controller.ts`

Antes: Siempre respondía `ok` sin verificar conectividad

Después: Valida conexión real a BD

```

@Get()
async check() {
  try {
    // PREVENTIVO: Valida conexión real a la BD
    await this.createRepo.query('SELECT 1');
    return {
      status: 'ok',
      timestamp: new Date().toISOString(),
      database: 'connected',
    };
  } catch (error) {
    throw new HttpException(
      {
        status: 'error',
        message: 'Database connection failed',
        timestamp: new Date().toISOString(),
      },
      HttpStatus.SERVICE_UNAVAILABLE,
    );
  }
}

```

Justificación

Estos cambios clasifican como **Mantenimiento Preventivo** porque:

- ☒ Reducen riesgo futuro de corrupción de datos
- ☒ Evitan crash por payloads inválidos
- ☒ Detectan fallos de conectividad antes de que afecten usuarios
- ☒ Implementan defensa en profundidad contra entrada malformada
- ☒ Mejoran observabilidad operativa

5. CAMBIO ESTRUCTURAL - Inconsistencia en DeleteEventEntity

Problema: DeleteEventEntity no tenía campo description mientras que otras entidades sí

Solución: Se agregó el campo para mantener consistencia

```

@Column({ nullable: true })
description: string;

```

Resumen de Cambios

Tipo	Archivo	Líneas	Cambio
Correctivo	events.service.ts	58-66	Agregar <code>await save()</code> en DELETE
Adaptativo	events.service.ts	30+	Cambiar a <code>toISOString()</code>
Perfectivo	events.service.ts	75-107	Normalizar timestamps en findAll
Perfectivo	events.service.ts	116-130	Incluir query_events en stats
Preventivo	create-event.dto.ts	1-40	Agregar validadores
Preventivo	main.ts	1-10	Agregar ValidationPipe global
Preventivo	health.controller.ts	1-30	Validar conexión BD
Estructural	delete-event.entity.ts	17-19	Agregar field description

Pruebas Recomendadas

1. Verificar DELETE se guarda

```
curl -X POST http://localhost:3000/events \
-H "Content-Type: application/json" \
-d '{
  "source": "mi-crud",
  "entity": "pedido",
  "action": "DELETE",
  "title": "Pedido eliminado",
  "description": "Pedido #123 fue eliminado",
  "payload": {"id": 123}
}'

# Verificar que se guardó
curl http://localhost:3000/events
```

2. Verificar formato de fechas ISO

```
# Las fechas en respuesta deben ser ISO: "2026-05-06T10:30:45.123Z"
```

3. Verificar stats completo

```
curl http://localhost:3000/stats
# Debe incluir: { create, update, delete, query, total }
```

4. Verificar validaciones

```
# Debe fallar (title muy largo)
curl -X POST http://localhost:3000/events \
-H "Content-Type: application/json" \
-d '{
  "source": "test",
  "entity": "test",
  "action": "CREATE",
  "title": "Este es un título extraordinariamente largo que excede el límite de 255 caracteres permitidos por el validador de class-validator",
  "payload": {}
}'
```

5. Verificar health check

```
curl http://localhost:3000/health
# Debe mostrar { status: 'ok', database: 'connected', timestamp: ISO }
```

Conclusión

El sistema EPN Event Manager ahora es más robusto, adaptable y mantenible. Los cambios aplicados cubren los 4 tipos de mantenimiento:

1. **Correctivo:** Bugs corregidos
2. **Adaptativo:** Migración a estándares internacionales
3. **Perfectivo:** Mejora de calidad y completitud
4. **Preventivo:** Defensa proactiva contra fallos

El sistema mantiene su propósito educativo (demostrar antipatrones de diseño) mientras reduce riesgos operativos.